

Model-Based Torque Control and Dynamics Validation of the Unitree Z1 Robot Arm

Computed PID Control, SDK/Gazebo Comparison, and Friction-Limited Real-Hardware Testing

Chunsheng Zeng | Student ID: 999012867

June 2026

Abstract

This project develops and evaluates a model-based torque-control framework for the Unitree Z1 robot arm. The main focus is the construction, validation, and use of a gripper-equivalent analytic dynamics model for computed PID torque control. The dynamics model follows the open-chain manipulator formulation from Murray, Li, and Sastry and computes the mass matrix $M(q)$, the Coriolis matrix $C(q, \dot{q})$, the gravity vector $N(q)$, and the derivative $\partial M / \partial q$. Because the Z1 final-link assumption strongly affects inverse-dynamics torque, the project uses an SDK/gripper-equivalent final-link model rather than the no-gripper Gazebo URDF final link.

The analytic dynamics implementation was compared with Unitree SDK `inverseDynamics`, and the mass, gravity, velocity-dependent, and full torque terms were found to be very close under the matched gripper-equivalent assumption. This supports the use of the model for computed-torque and computed-PID control. In Gazebo simulation, friction and damping compensation were required to obtain small tracking error, showing that the rigid-body model alone is not sufficient when actuator resistance and deadband are present. The Unitree SDK `LOWCMD` path was also analyzed as a reference because it combines feedforward torque with internal position and damping feedback, while the project's pure torque bridge applies all feedback externally.

Representative real-arm tests showed that the physical Z1 arm has stronger friction/deadband and safety constraints than the simplified simulation model. The computed PID controller with friction compensation was able to perform meaningful real-arm motion, but larger forward-pose tests showed residual error and possible oscillation risk when friction compensation was increased. Therefore, real-hardware testing was stopped for safety, and the remaining systematic comparison was completed using Gazebo and SDK/LOWCMD logs. Overall, the project shows that the gripper-equivalent analytic dynamics model is consistent with SDK and simulation behavior, while the main practical limitation for real torque control is friction/deadband identification, feedback bandwidth, and safe torque authority.

Code availability

The code for this project is publicly available on GitHub:

https://github.com/iceswordIV/torque_control

The repository contains the analytic dynamics implementation, controller code, comparison scripts, log-analysis scripts, and documentation used to generate the results in this report. The main dynamics implementation is in `z1_project/z1_analytic_dynamics.py`, and the controller implementations are in `z1_project/test_controller.py` and related control scripts.

Nomenclature

Symbol	Meaning
q	joint position vector
$\dot{q}$	joint velocity vector
$\ddot{q}$	joint acceleration vector
$q_d, \dot{q}_d, \ddot{q}_d$	desired joint position, velocity, and acceleration
$e = q_d - q$	position tracking error
$\dot{e} = \dot{q}_d - \dot{q}$	velocity tracking error
$M(q)$	manipulator mass / inertia matrix
$C(q, \dot{q})$	Coriolis / centrifugal matrix
$N(q)$	gravity torque vector
τ	joint torque command
$\xi = [v; \omega]$	twist coordinate vector in this project
$A d_g$	adjoint transformation associated with rigid transform g
CPID	computed PID torque controller
SDK LOWCMD	Unitree low-level command path using reference position, velocity, feedforward torque, and internal gains

1. Introduction

Torque control is important for robot arms because it gives direct access to the physical interaction between the controller and the mechanism. Unlike high-level position control, torque control requires a usable dynamics model and a careful understanding of actuator limitations. This project studies torque control on the Unitree Z1 robot arm, with particular attention to computed PID control using an analytic dynamics model.

The project is motivated by a practical problem: a model-based controller can be correct in theory, but still perform poorly if the model parameters, gripper assumption, friction compensation, or control path are not handled correctly. The Unitree Z1 arm used in this project has a gripper installed. Therefore, the distal mass and inertia seen by the upstream joints differ from a no-gripper arm model. This is especially important for computed torque and computed PID control, because these controllers use $M(q)$, $C(q, \dot{q})$, and $N(q)$ directly when calculating torque.

The project therefore has three main goals. First, it builds a gripper-equivalent analytic dynamics model for the Z1 arm. Second, it validates the model against Unitree SDK `inverseDynamics` and Gazebo behavior. Third, it implements computed PID torque control and studies the practical effects of friction, damping, deadband, torque limits, and control-path bandwidth.

The project does not only compare controller labels. The augmented-PD/friction controller and SDK `LOWCMD` are used as references to understand the behavior of Gazebo and the difference between pure external torque control and Unitree's internal low-level feedback path. The central technical work remains the gripper-equivalent dynamics model and the CPID torque-control framework.

The main contributions are:

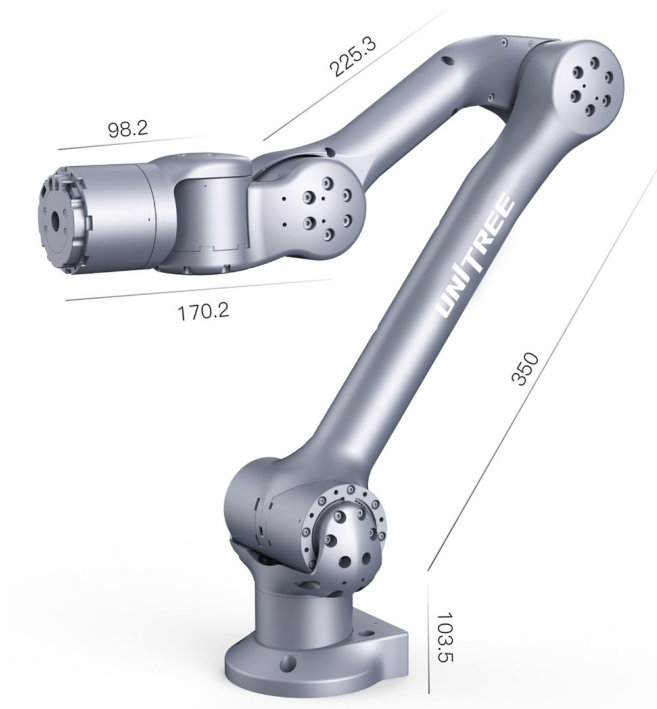
1. A gripper-equivalent analytic dynamics model was built for the Unitree Z1 arm. Links 1-5 use the Z1 URDF parameters, while the final link uses an SDK/gripper-equivalent rigid-body assumption.
2. The analytic dynamics implementation was mapped directly to open-chain robot dynamics formulas from Murray, Li, and Sastry.
3. The Python analytic dynamics model was compared with Unitree SDK `inverseDynamics`, showing close agreement when the gripper-equivalent final-link assumption is matched.
4. A computed PID torque controller with friction/damping compensation was implemented and tested.
5. The project analyzed why friction compensation is required and why excessive compensation can create oscillation or motor heating.
6. The project compared pure external torque control with SDK `LOWCMD` and explained why these two command paths are not equivalent.
7. Real-hardware tests were performed conservatively to identify practical transfer limitations from Gazebo to the physical Z1 arm.

2. Unitree Z1 Platform and Control Architecture

2.1 Platform

The Unitree Z1 is a six-degree-of-freedom robot arm. In this project, the real arm includes the gripper. The presence of the gripper is important because it increases the distal mass and changes the gravity and inertia terms seen by the upstream joints. For this reason, the analytic

model used in this report is a gripper-equivalent dynamics model rather than a no-gripper model.



Unitree Z1 arm dimensions and link geometry used as the physical platform reference.

2.2 Pure external torque control path

The project pure-torque path computes the complete joint torque externally. The controller computes the model terms, feedback terms, and friction/damping compensation, and then sends only the final torque command. In this path, the low-level position and damping gains are set to zero:

$$K_{p_low} = 0$$

$$K_{d_low} = 0$$

Therefore, all feedback is applied in the external loop. This makes the system sensitive to Python computation time, communication delay, ROS/file bridge timing, feedback sampling rate, velocity noise, and command jitter.

2.3 SDK LOWCMD path

The SDK LOWCMD path is structurally different. It sends desired joint position, desired velocity, feedforward torque, and low-level gains. The effective structure is

$$\tau_{applied} = \tau_{ff} + K_p(q_{cmd} - q) + K_d(\dot{q}_{cmd} - \dot{q}).$$

This means SDK LOWCMD is not pure feedforward torque. It applies internal position and damping feedback closer to the plant. Therefore, replaying only the SDK feedforward torque through the pure external torque bridge is not equivalent to SDK LOWCMD.

2.4 Why architecture matters

The architecture difference explains why SDK LOWCMD can track better even when the feedforward torque alone is modest. In SDK LOWCMD, feedback is applied inside the Unitree/Gazebo path and at a higher effective bandwidth. In the pure external torque path, the feedback must be computed outside the plant and sent through the bridge.

This report therefore uses SDK LOWCMD as a reference for architecture and achievable behavior, not as a pure torque replay baseline.

3. Gripper-Equivalent Unitree Z1 Dynamics Model

3.1 Why the gripper-equivalent model is used

The dynamics model used in this project is a gripper-equivalent Unitree Z1 model. This choice is necessary because the physical Z1 arm used in the real-hardware experiments has the gripper installed. The gripper changes the distal mass, center of mass, and inertia seen by the six arm joints. Therefore, using a no-gripper final-link model would underestimate the torque required by the upstream joints, especially for gravity compensation and computed PID torque control.

The z1_description package contains both the standard arm parameters and optional gripper parameters. In the generated no-gripper Gazebo URDF, the last arm link link06 has a mass of only 0.28875807 kg. When the optional gripper stator and gripper mover are considered, the distal mass becomes much larger. In this project, the final distal assembly is represented by one SDK/gripper-equivalent rigid body. The final-link mass used in the analytic model is 1.09473306147355 kg.

This means the project dynamics model should not be interpreted as the no-gripper Gazebo URDF model. Instead, it is a gripper-equivalent Z1 model intended to match the real arm and the Unitree SDK inverse-dynamics behavior more closely.

3.2 Coordinate and parameter conventions

Quantity	Convention
Mass	kg
COM position	meters, expressed in each link frame
Inertia tensor	kg m ² , about each link COM, expressed in the link frame
Joint axis w_i	expressed in the base frame at zero configuration
Joint point q_i	point on the joint axis, expressed in the base frame at zero configuration

Quantity	Convention
Twist convention	$\xi_i = [v_i; \omega_i]$, where $v_i = -\omega_i \times q_i$

3.3 Main analytic link parameters

Links 1-5 use the same mass, COM, and inertia values as the Z1 URDF model. The final link is replaced by an SDK/gripper-equivalent final rigid body.

Project link	Mass [kg]	COM xyz [m]	Inertia tuple ($I_{xx}, I_{xy}, I_{xz}, I_{yy}, I_{yz}, I_{zz}$) [kg m ²]
link01	0.67332551	(2.47e-06, -0.00025198, 0.02317169)	(0.00128328, -6e-08, -4e-07, 0.00071931, 5e-07, 0.00083936)
link02	1.19132258	(-0.11012601, 0.00240029, 0.00158266)	(0.00102138, 0.00062358, 5.13e-06, 0.02429457, -2.1e-06, 0.02466114)
link03	0.83940874	(0.10609208, -0.00541815, 0.03476383)	(0.00108061, -8.669e-05, -0.00208102, 0.00954238, -1.332e-05, 0.00886621)
link04	0.56404563	(0.04366681, 0.00364738, -0.00170192)	(0.00031576, 8.13e-05, 4.091e-05, 0.00092996, -5.96e-06, 0.00097912)
link05	0.38938492	(0.03121533, 0, 0.00646316)	(0.00017605, 4e-07, 5.689e-05, 0.00055896, -1.3e-07, 0.00053860)
link06 equivalent	1.09473306147355	(0.023210304542, -0.000363250494, 0.002026681669)	(0.00336754278998 6, -1.884722331677e-0 5, 0.00028294370740 01, 0.002529152661291, 2.211249714817e-05,

Project link	Mass [kg]	COM xyz [m]	Inertia tuple ($I_{xx}, I_{xy}, I_{xz}, I_{yy}, I_{yz}, I_{zz}$) [kg m ²]
			0.002713697251443)

The final row is the most important difference from the no-gripper URDF model. The no-gripper generated URDF uses link06 = 0.28875807 kg, while the analytic model uses link06 equivalent = 1.09473306147355 kg.

3.4 Joint axes and joint points

Joint	Axis w_i	Base-frame joint point q_i [m]
J1	(0, 0, 1)	(0, 0, 0.0585)
J2	(0, 1, 0)	(0, 0, 0.1035)
J3	(0, 1, 0)	(-0.35, 0, 0.1035)
J4	(0, 1, 0)	(-0.132, 0, 0.1605)
J5	(0, 0, 1)	(-0.062, 0, 0.1605)
J6	(1, 0, 0)	(-0.0128, 0, 0.1605)

For each revolute joint, the twist is constructed as

$$\xi_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}, v_i = -\omega_i \times q_i.$$

3.5 Distal-link comparison

Distal model	Components	Mass [kg]	Meaning
No-gripper Gazebo URDF	link06 only	0.28875807	Generated URDF when UnitreeGripper = false
Simple optional-gripper sum	link06 + gripperStator + gripperMover	1.09100764	Direct mass sum using optional xacro gripper bodies
Project analytic model	SDK/gripper-equivalent link06	1.09473306147355	Fitted equivalent rigid body used for analytic dynamics

The simple optional-gripper mass is close to the project final-link mass. However, the project final link is not a direct rigid concatenation of the URDF gripper bodies. Its COM and inertia are fitted equivalent parameters chosen to match the Unitree SDK inverseDynamics behavior.

3.6 Why the parameter choice matters for CPID

The computed PID controller directly uses the rigid-body dynamics model:

$$\ddot{q}_{cmd} = \ddot{q}_d + K_d(\dot{q}_d - \dot{q}) + K_p(q_d - q) + K_i \int (q_d - q) dt,$$
$$\tau = M(q)\ddot{q}_{cmd} + C(q, \dot{q})\dot{q} + N(q) + \tau_{fric}.$$

Because $M(q)$, $C(q, \dot{q})$, and $N(q)$ are computed from the link parameters, the gripper assumption directly affects the commanded torque. The final-link mass and inertia influence not only J6, but also the upstream joints J1-J5, because those joints support and accelerate the distal assembly.

4. Formula-to-Code Implementation of $M(q)$, dM/dq , $C(q, dq)$, and $N(q)$

4.1 Purpose

The computed PID torque controller depends directly on $M(q)$, $C(q, \dot{q})$, and $N(q)$. Therefore, the report must show not only the final controller equation, but also how these terms are constructed in code. This chapter maps the theoretical formulas from Murray, Li, and Sastry to the Python implementation used in this project.

4.2 Hat map and cross-product matrix

The hat map is defined in Eq. (2.4), and the cross product is written as matrix multiplication in Eq. (2.5):

$$a \times b = \hat{a} b.$$

The corresponding code is:

```
def skew3(a: np.ndarray) -> np.ndarray:
    a = np.asarray(a).reshape(3)
    return np.array(
        [
            [0.0, -a[2], a[1]],
            [a[2], 0.0, -a[0]],
            [-a[1], a[0], 0.0],
        ],
        dtype=np.result_type(a, float),
    )
```

4.3 Revolute twists and product of exponentials

For a revolute joint, the twist is constructed using a joint axis ω_i and a point q_i on the joint axis:

$$\xi_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}, v_i = -\omega_i \times q_i.$$

The project code constructs the Z1 twists as:

```
def z1_twists(p: Optional[Z1Params] = None):
    p = z1_parameters() if p is None else p
    w_list = p.w.copy()
    v_list = np.zeros((3, NDOF), dtype=float)
    twists = np.zeros((6, NDOF), dtype=float)

    for i in range(NDOF):
        w = w_list[:, i]
        q0 = p.q_axis[:, i]
        v = -np.cross(w, q0)
        v_list[:, i] = v
        twists[:, i] = np.r_[v, w]

    return twists, w_list, v_list
```

4.4 Twist exponential and adjoint transformation

The code implements the rigid-body exponential map using Rodrigues' formula and the twist exponential. The adjoint transformation follows Eq. (2.58):

$$Ad_g = \begin{bmatrix} R & \hat{p} R \\ 0 & R \end{bmatrix}.$$

```
def adjoint(g: np.ndarray) -> np.ndarray:
    R = g[:3, :3]
    p = g[:3, 3]
    Ad = np.zeros((6, 6), dtype=dtype)
    Ad[:3, :3] = R
    Ad[:3, 3:6] = skew3(p) @ R
    Ad[3:6, 3:6] = R
    return Ad
```

4.5 Mass matrix

The mass matrix is based on the link kinetic energy expressions in Eq. (4.17)-Eq. (4.19) and the product-of-exponentials explicit formula in Eq. (4.29):

$$M_{ij}(q) = \sum_{l=\max(i,j)}^n \xi_i^T A_{li}^T M'_l A_{lj} \xi_j.$$

The implementation constructs the link Jacobian columns and then sums the reflected inertias:

```

J = np.zeros((NDOF, NDOF, 6), dtype=float)
for l in range(NDOF):
    for j in range(l + 1):
        J[l, j] = A[l, j] @ xi[:, j]

GJ = np.einsum("lab,ljb->lja", Gp, J)
M = np.zeros((NDOF, NDOF), dtype=float)
for i in range(NDOF):
    for j in range(NDOF):
        val = 0.0
        for l in range(max(i, j), NDOF):
            val += J[l, i] @ GJ[l, j]
        M[i, j] = val
M = 0.5 * (M + M.T)

```

4.6 Mass-matrix derivative and Coriolis matrix

The derivative of the mass matrix is calculated from the derivative of each link Jacobian column. This follows the open-chain proof used in Murray, Li, and Sastry. For $i \geq j$, define

$$g_{ij} = \begin{cases} e^{-\hat{\xi}_i \theta_i} \dots e^{-\hat{\xi}_{j+1} \theta_{j+1}}, & i > j, \\ I, & i = j, \end{cases}$$

and

$$A_{ij} = Ad_{g_{ij}}.$$

For $i \geq k \geq j$, the derivative of one body-Jacobian column is

$$\frac{\partial}{\partial \theta_k} (A_{ij} \xi_j) = A_{ik} [A_{kj} \xi_j, \xi_k].$$

For all other values of k , the derivative is zero. This is exactly the rule used in the project code:

```

def dJ_column_analytic(l: int, j: int, k: int, A: np.ndarray, xi: np.ndarray) -> np.ndarray:
    if not (l >= k >= j):
        return np.zeros(6)
    return A[l, k] @ lie_bracket_vw(A[k, j] @ xi[:, j], xi[:, k])

```

After this, the mass matrix

$$M_{ij}(q) = \sum_{l=\max(i,j)}^n J_{li}^T M'_l J_{lj}$$

is differentiated directly:

$$\frac{\partial M_{ij}}{\partial q_k} = \sum_{l=\max(i,j)}^n \left(\frac{\partial J_{li}^T}{\partial q_k} M'_{lJ_{lj}} + J_{li}^T M'_{lJ_{lj}} \frac{\partial J_{lj}}{\partial q_k} \right).$$

The code implements the same two-term product-rule derivative:

```
dM = np.zeros((NDOF, NDOF, NDOF), dtype=float)
for k in range(NDOF):
    for i in range(NDOF):
        for j in range(NDOF):
            val = 0.0
            for l in range(max(i, j), NDOF):
                val += dJ[l, i, k] @ GJ[l, j] + J[l, i] @ GdJ[l, j, k]
            dM[i, j, k] = val
```

The Coriolis matrix is then constructed from the Christoffel expression:

$$C_{ij}(q, \dot{q}) = \frac{1}{2} \sum_k \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{kj}}{\partial q_i} \right) \dot{q}_k.$$

```
def coriolis_from_dM(dM: np.ndarray, dq: np.ndarray) -> np.ndarray:
    C = np.zeros((NDOF, NDOF), dtype=float)
    for i in range(NDOF):
        for j in range(NDOF):
            C[i, j] = 0.5 * sum(
                (dM[i, j, k] + dM[i, k, j] - dM[k, j, i]) * dq[k]
                for k in range(NDOF)
            )
    return C
```

Therefore, the project does not approximate the Coriolis term by tuning. It computes dM/dq analytically through the Lie-bracket Jacobian derivative, and then forms C(q,dq) from the standard Christoffel formula.

4.7 Gravity vector and sign convention

The gravity term needs a careful sign convention. In this report, the manipulator equation is written as

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + N(q) = \tau.$$

With this convention, $N(q)$ is the actuator torque required to compensate gravity. If the potential energy is

$$V(q) = \sum_i m_i g h_i(q),$$

then

$$N(q) = \frac{\partial V(q)}{\partial q}.$$

The physical gravity generalized force has the opposite sign:

$$Q_g = J_i^T F_{w_i}^b = - \frac{\partial V_i}{\partial q}.$$

Equivalently,

$$\frac{\partial V_i}{\partial q} = - J_i^T F_{w_i}^b.$$

This is why the negative sign appears in the virtual-work form when the gravity wrench symbol denotes the actual downward gravitational force. It does not mean that the gravity compensation term in the actuator equation should use the negative gradient. In this project, the controller uses the compensation convention $N = dV/dq$.

The implementation includes a finite-difference check of dV/dq :

```
def potential_energy(q: np.ndarray) -> float:
    p = z1_parameters()
    p_com = link_com_positions(q)
    return float(np.sum(p.m_vec * p.g_const * p_com[2, :]))

def gravity_vector_finite_difference(q: np.ndarray, step: float = 1e-6) -> np.ndarray:
    ...
    N[j] = (potential_energy(q_plus) - potential_energy(q_minus)) / (2.0 * h)
    return N
```

The main Jacobian-based gravity function computes the same compensation torque directly from COM motion:

```
def gravity_vector(q: np.ndarray) -> np.ndarray:
    ...
    for j in range(NDOF):
        vj = J_space[:, 3, j]
        wj = J_space[3:6, j]
        for l in range(j, NDOF):
            dp = vj + np.cross(wj, p_com[:, l])
            N[j] += p.m_vec[l] * p.g_const * dp[2]
    return N
```

Here, $dp[2]$ equals dz_l/dq_j , so the summation gives

$$N_j(q) = \sum_l m_l g \frac{\partial z_l}{\partial q_j} = \frac{\partial V}{\partial q_j}.$$

The code also contains a virtual-work implementation using the body-frame gravity wrench. That function explicitly writes the convention

$$\frac{\partial V}{\partial q} = -J_{body}^T F_{gravity, body}.$$

Thus the potential-energy, Jacobian, and virtual-work interpretations are consistent when the sign convention is stated correctly.

4.8 Full analytic dynamics wrapper

The controller uses:

```
def dynamics_analytic(q: np.ndarray, dq: np.ndarray):
    M, dM = mass_and_dM_analytic(q)
    C = coriolis_from_dM(dM, dq)
    N = gravity_vector(q)
    return M, C, N, dM
```

This function returns the complete dynamics set needed by computed PID:

$$(M, C, N, dM) = \text{dynamics_analytic}(q, \dot{q}).$$

4.9 Cython acceleration

The analytic calculation of $M(q)$, dM/dq , $C(q, \dot{q})$, and $N(q)$ is computationally expensive. The dynamics module therefore supports an optional compiled accelerator:

```
try:
    if os.environ.get("Z1_DISABLE_FAST_DYNAMICS"):
        _FAST_DYNAMICS = None
    else:
        import z1_analytic_dynamics_fast as _FAST_DYNAMICS
except ImportError:
    _FAST_DYNAMICS = None
```

Cython acceleration improves model computation time. It does not remove communication delay, feedback delay, bridge delay, actuator friction, or motor safety limits.

4.10 Formula-to-code mapping summary

Theory / formula	Book equation	Code function
Hat map	Eq. (2.4)	skew3()
Cross product as matrix multiplication	Eq. (2.5)	skew3(a) @ b
Rotation exponential	Eq. (2.9), Eq. (2.14)	rodrigues()
Revolute twist	Eq. (2.26), Eq. (2.30), Eq. (2.31)	z1_twists()

Theory / formula	Book equation	Code function
Rigid-body exponential	Eq. (2.36)	twist_exp()
Adjoint transformation	Eq. (2.58)	adjoint(), adjoint_inverse()
Link kinetic energy	Eq. (4.17)	link Jacobian and spatial inertia calculation
Total kinetic energy	Eq. (4.18)	mass-matrix construction
Manipulator mass matrix	Eq. (4.19), Eq. (4.29)	mass_and_dM_analytic()
Lie bracket	Eq. (4.26)	lie_bracket_vw()
Adjoint chain	Eq. (4.27)	compute_A_and_Gp()
Transformed inertia	Eq. (4.28)	Gp[l] = A0.T @ G @ A0
Mass-matrix derivative	Eq. (4.30)	dJ_column_analytic(), dM loop
Christoffel / Coriolis terms	Eq. (4.22)	coriolis_from_dM()
Gravity vector	$V(q) = \sum_i m_i g h_i(q)$	potential_energy(), gravity_vector()
Full dynamics	Eq. (4.47)	dynamics_analytic()

5. Dynamics Model Validation Against Unitree SDK

5.1 Purpose of validation

The computed PID controller uses the analytic dynamics model directly. Before discussing controller results, the dynamics model must be validated. The goal is to check whether the Python analytic dynamics calculation is consistent with Unitree SDK inverseDynamics when the same gripper-equivalent final-link assumption is used.

5.2 SDK inverseDynamics as reference

The Unitree SDK provides an inverseDynamics calculation through the Z1 arm model. It is a useful reference because it represents the manufacturer-supported model. In this project, the comparison is performed using the gripper-equipped assumption, because the real arm has the gripper installed.

The correct comparison is:

Python gripper-equivalent analytic model vs. Unitree SDK inverseDynamics with gripper .

5.3 Static forward-pose torque comparison

A representative validation was performed at

$$q=[0, 1.5, -1, -0.54, 0, 0],$$

with zero velocity and zero acceleration. This isolates the gravity-dominated inverse-dynamics torque.

The gripper-equipped comparison gave approximately:

$$\tau_{SDK} \approx [0, -6.6688, -7.4410, -2.1574, 0.00007, -0.00185],$$

$$\tau_{Python} \approx [0, -6.6752, -7.4488, -2.1537, 0.00016, -0.00390].$$

The two torque vectors have the same sign, same scale, and close numerical values. This supports the use of the gripper-equivalent analytic model for model-based torque control.

5.4 Full local dynamics comparison

Compared quantity	Maximum difference
$\max M_{python} - M_{sdk} $	0.00340829
$\max N_{python} - N_{sdk} $	0.0104184
$\max \ h_{python} - h_{sdk}\ $	0.00104424
$\max \ \tau_{python} - \tau_{sdk}\ $	0.0143673
$\max dM_{python} - ddM_{sdk} $	0.0080013

These values show close agreement between the Python analytic model and the Unitree SDK inverseDynamics calculation.

5.5 Validation in Gazebo simulation

The SDK comparison validates the local inverse-dynamics calculation. The Gazebo experiments then check whether the same model is useful inside a torque-control loop. The simulation results show that the model-based torque framework can track trajectories with small error after friction and damping compensation are included. This indicates that the rigid-body terms are sufficient to support motion, while the remaining tracking error is strongly affected by non-ideal friction and damping effects.

5.6 SDK LOWCMD is not pure feedforward torque

SDK LOWCMD should not be interpreted as pure feedforward torque. Its effective structure is

$$\tau_{applied} = \tau_{ff} + K_p(q_{cmd} - q) + K_d(\dot{q}_{cmd} - \dot{q}).$$

The feedforward term is only one part of the applied torque. Therefore, SDK LOWCMD is a reference for achievable behavior and control architecture, not a pure torque replay baseline.

6. Computed PID Torque Controller

6.1 Purpose

The main model-based controller studied in this project is computed PID torque control. The controller uses the validated gripper-equivalent dynamics model to convert a desired joint-space acceleration into a physical torque command.

6.2 Error convention

This project uses

$$e = q_d - q, \dot{e} = \dot{q}_d - \dot{q}.$$

With this convention, stabilizing feedback appears with positive gains:

$$K_p e + K_d \dot{e}.$$

6.3 Computed PID acceleration command

The computed PID controller first constructs the commanded acceleration:

$$\ddot{q}_{cmd} = \ddot{q}_d + K_d (\dot{q}_d - \dot{q}) + K_p (q_d - q) + K_i \int (q_d - q) dt.$$

The implementation is:

```
e = q_des - q
de = dq_des - dq
i_accel = Ki @ e_int
ddq_cmd = ddq_des + Kd @ de + Kp @ e + i_accel
```

6.4 Inverse-dynamics torque calculation

After $\ddot{q}_{cmd}$ is constructed, the controller computes

$$\tau_{model} = M(q) \ddot{q}_{cmd} + C(q, \dot{q}) \dot{q} + N(q).$$

```
M, C, N, _ = _dynamics_for_mode(q, dq, ...)
```

```
tau_model = M @ ddq_cmd + C @ dq + N
```

6.5 Relation to computed torque control

The standard computed torque controller uses

$$\tau = M(q) (\ddot{q}_d + K_d \dot{e} + K_p e) + C(q, \dot{q}) \dot{q} + N(q),$$

which ideally gives

$$\ddot{e} + K_d \dot{e} + K_p e = 0.$$

The project CPID controller extends this by adding the integral acceleration term $K_i e_{int}$.

6.6 CPID with friction and damping compensation

The implemented computed PID with friction compensation is

$$\tau = M(q)\ddot{q}_{cmd} + C(q, \dot{q})\dot{q} + N(q) + D\dot{q}_d + F \cdot direction.$$

```
tau_i, tau_model = compute_computed_pid_model_components(...)
```

```
tau_damping, tau_friction = _gazebo_friction_terms(...)
```

```
tau = tau_model + tau_damping + tau_friction
```

6.7 Torque limits and safety

The real-arm controller applies torque limits to prevent unsafe motion. A typical real test used

$$\tau_{max} = [5, 12, 12, 10, 3, 3] \text{ Nm}.$$

These limits protect the hardware but also limit the controller's ability to overcome friction and deadband.

7. Friction and Damping Compensation

7.1 Why compensation is separate from rigid-body dynamics

The ideal model is

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + N(q) = \tau.$$

A more practical actuator-level torque balance is

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + N(q) + \tau_{friction} + \tau_{damping} + \tau_{deadband} = \tau_{motor}.$$

Therefore, the controller command is extended to

$$\tau_{cmd} = \tau_{rigid} + \tau_{comp}, \tau_{comp} = \tau_{damping} + \tau_{friction}.$$

7.2 Damping compensation

The damping compensation used in this project is

$$\tau_{damping} = D\dot{q}_d.$$

The desired velocity is smoother than raw measured velocity and follows the planned S-curve trajectory.

7.3 Coulomb friction compensation and deadband logic

The dry-friction compensation is modeled as

$$\tau_{friction} = F \cdot direction \cdot \text{wants_motion}.$$

When desired velocity is large enough, the direction is chosen from $\text{sign}(\dot{q}_d)$. Near zero desired velocity, the direction is chosen from $\text{sign}(q_d - q)$.

The implementation is:

```
tau_damping = damping_values * dq_des
velocity_direction = np.sign(dq_des)
error_direction = np.sign(e)

direction = np.where(
    np.abs(dq_des) > FRICTION_VELOCITY_EPS,
    velocity_direction,
    error_direction
)

wants_motion = (
    (np.abs(e) > friction_deadband)
    | (np.abs(dq_des) > FRICTION_VELOCITY_EPS)
)

tau_friction = friction_values * direction * wants_motion
```

7.4 Why compensation improves tracking

Friction and damping compensation are required because part of the commanded torque is consumed by simulated or physical joint resistance. In Gazebo, adding compensation greatly reduces tracking error. In real hardware, compensation is even more important because the physical arm includes gearbox friction, motor current behavior, deadband, cable effects, and safety filtering.

7.5 Why too much compensation is dangerous

If the friction value F is too small, the joint may not overcome static friction. If F is too large, the joint can overshoot after breakaway and then oscillate or chatter near the target. The sign-dependent friction term is discontinuous around zero velocity, so large compensation can cause heating and unsafe motion.

7.6 Future identification method

At near-zero velocity,

$$\dot{q} \approx 0, \ddot{q} \approx 0,$$

so

$$\tau_{\text{residual}} \approx \tau_{\text{cmd}} - N(q).$$

At constant velocity,

$$\tau_{residual}(v) \approx F_c \text{sign}(v) + Bv + \tau_{offset}.$$

This motivates PI-based breakaway tests and constant-velocity damping identification.

8. Simulation Benchmark and SDK LOWCMD Reference

8.1 Purpose

The simulation and SDK benchmark is used as a diagnostic tool, not as a simple controller-ranking exercise. It checks whether Gazebo can be controlled accurately when model feedforward and friction/damping compensation are included, compares CPID with reference behaviors, and clarifies the difference between pure external torque and SDK LOWCMD.

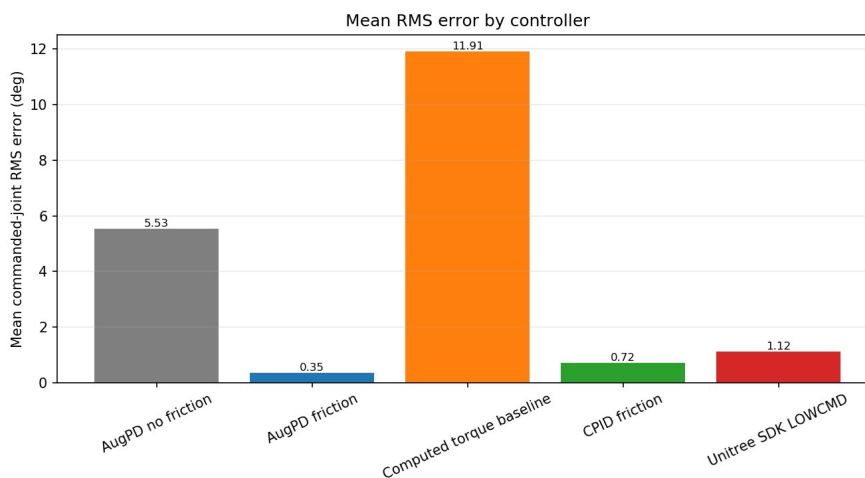
8.2 Test design

The single-joint benchmark covers commanded relative motions across all six joints. The comparison uses outbound-phase tracking metrics: achieved percent, final error, maximum absolute error, RMS error, and maximum torque. The comparison report used 150 single-joint rows: 120 pure-torque rows and 30 SDK LOWCMD reference rows. It also included 20 forward-pose rows: 16 pure-torque rows and 4 SDK LOWCMD reference rows.

8.3 Main single-joint result

Contro ller	Runs	Mean max error [deg]	Mean RMS error [deg]	Mean final error [deg]	Achieved [%]	Worst max error [deg]	Rate [Hz]
Augme nted PD, frictio n/dam ping	30	1.30	0.86	1.03	95.6	16.16	377.2
Augme nted PD, no frictio n	30	6.66	4.65	6.21	42.0	17.18	381.8
Comp uted torque , frictio n/dam ping	30	5.61	3.93	5.36	87.7	29.46	383.3
Comp	30	14.15	9.08	14.02	3.5	30.13	383.9

Controller	Runs	Mean max error [deg]	Mean RMS error [deg]	Mean final error [deg]	Achieved [%]	Worst max error [deg]	Rate [Hz]
Controlled torque, no friction							
Unitree SDK LOWCMD	30	1.11	0.67	0.47	96.7	7.74	491.2

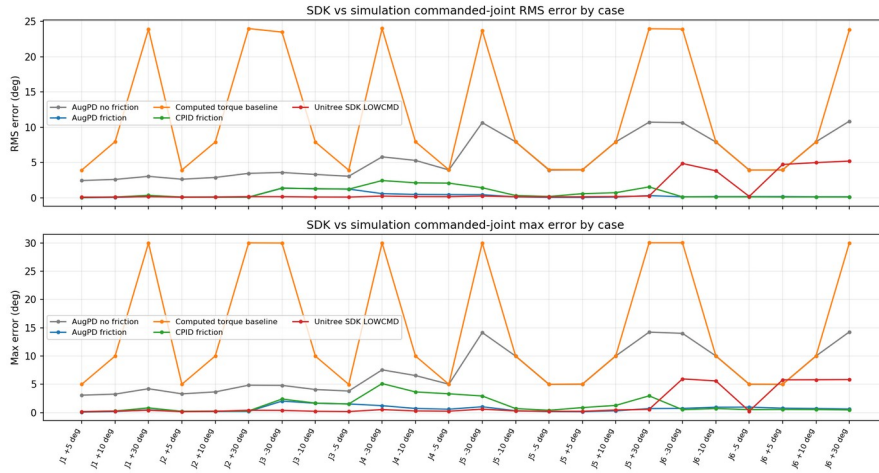


Mean RMS tracking error by controller.

This table should not be used to dismiss CPID. It shows that Gazebo requires friction/damping compensation and that CPID is more sensitive to timing, feedback, torque limits, and compensation accuracy under the external torque path.

8.4 SDK LOWCMD interpretation

SDK LOWCMD is not pure torque. It sends desired position, desired velocity, feedforward torque, and internal gains. Therefore, SDK can generate applied torque that differs from feedforward torque alone.



SDK versus simulation error comparison.

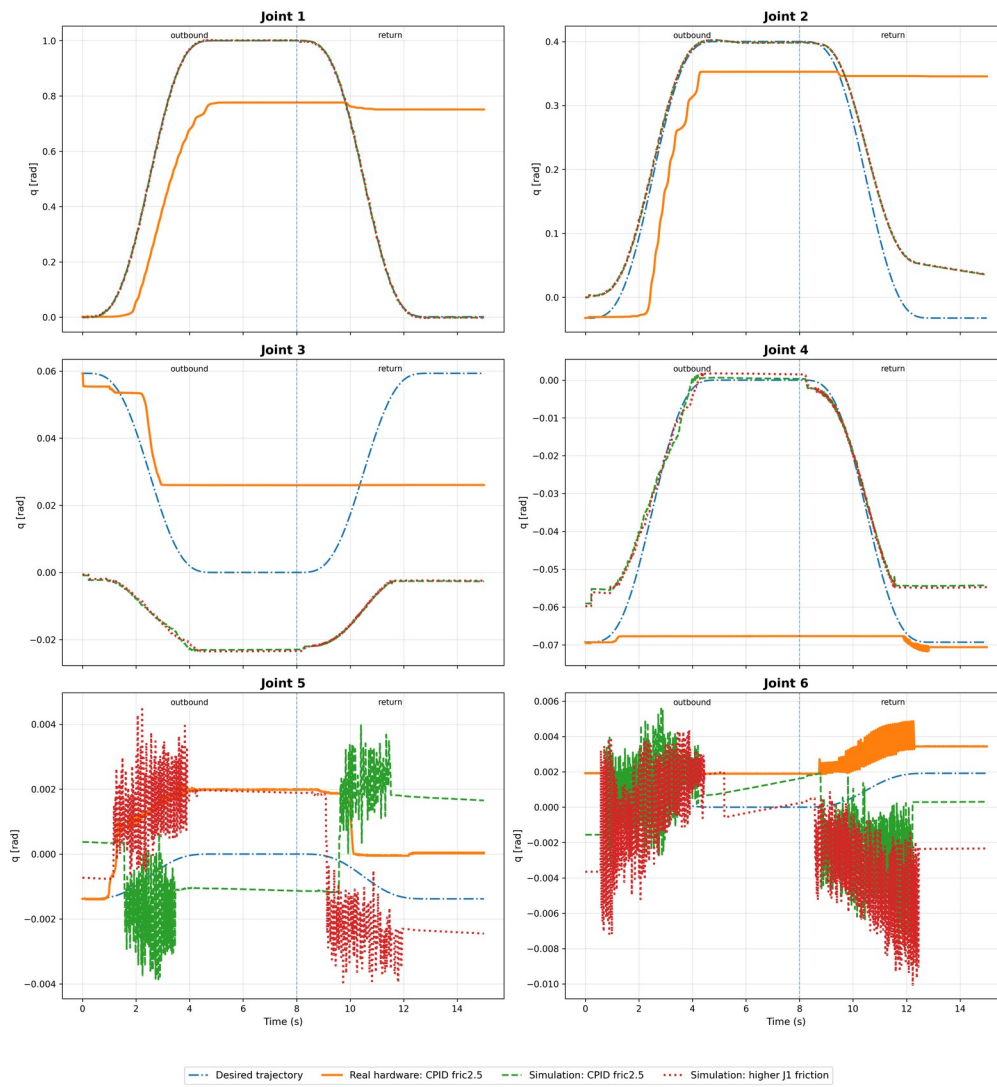
8.5 J4 diagnostic result

J4 shows direction-dependent behavior. Negative J4 can be tracked well by the pure-torque reference controller, while positive J4 remains difficult for both SDK and pure torque. This indicates direction-dependent plant/control behavior involving gravity, friction, coupling, saturation, low inertia, and external-loop delay.

8.6 Three-case forward-pose comparison

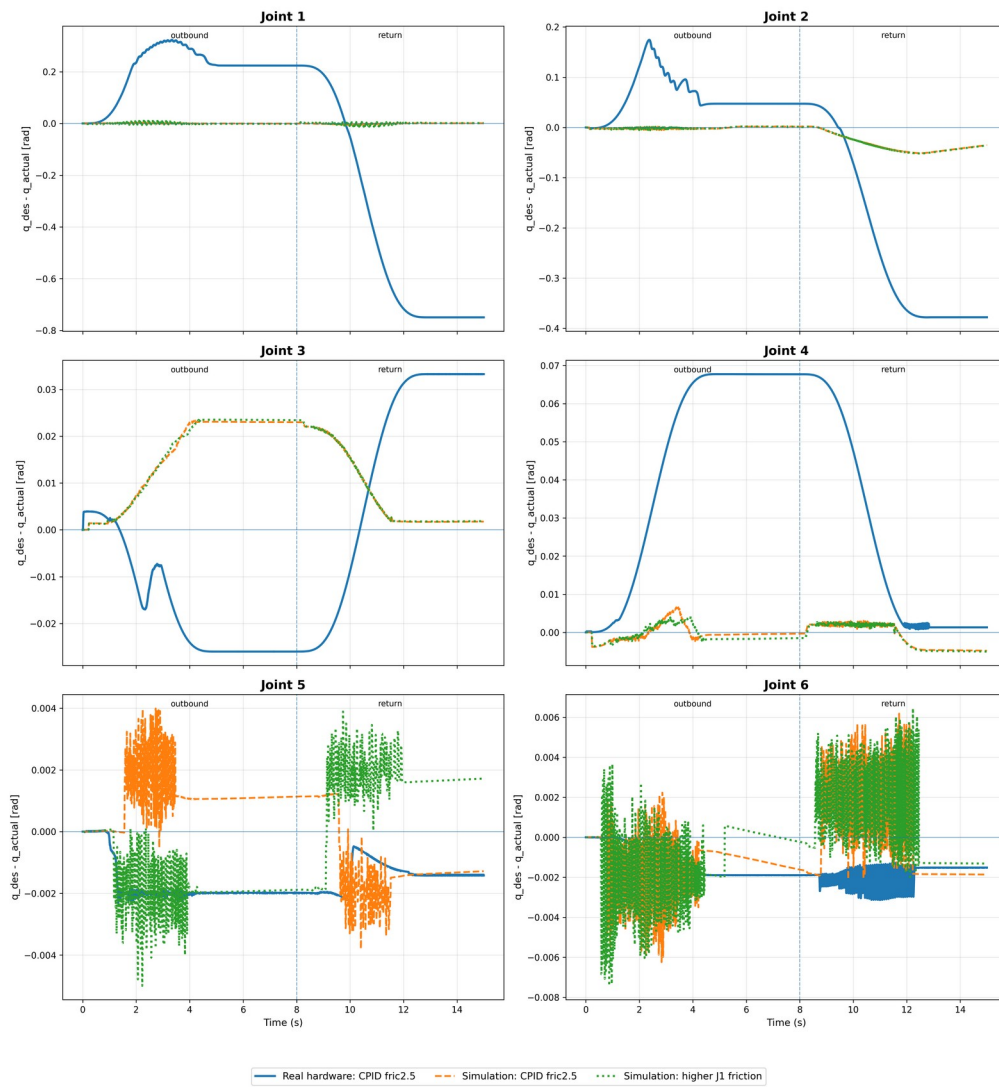
The three-case comparison visualizes real and simulation behavior under different friction assumptions.

Forward-Pose CPID Test: Joint Position Comparison



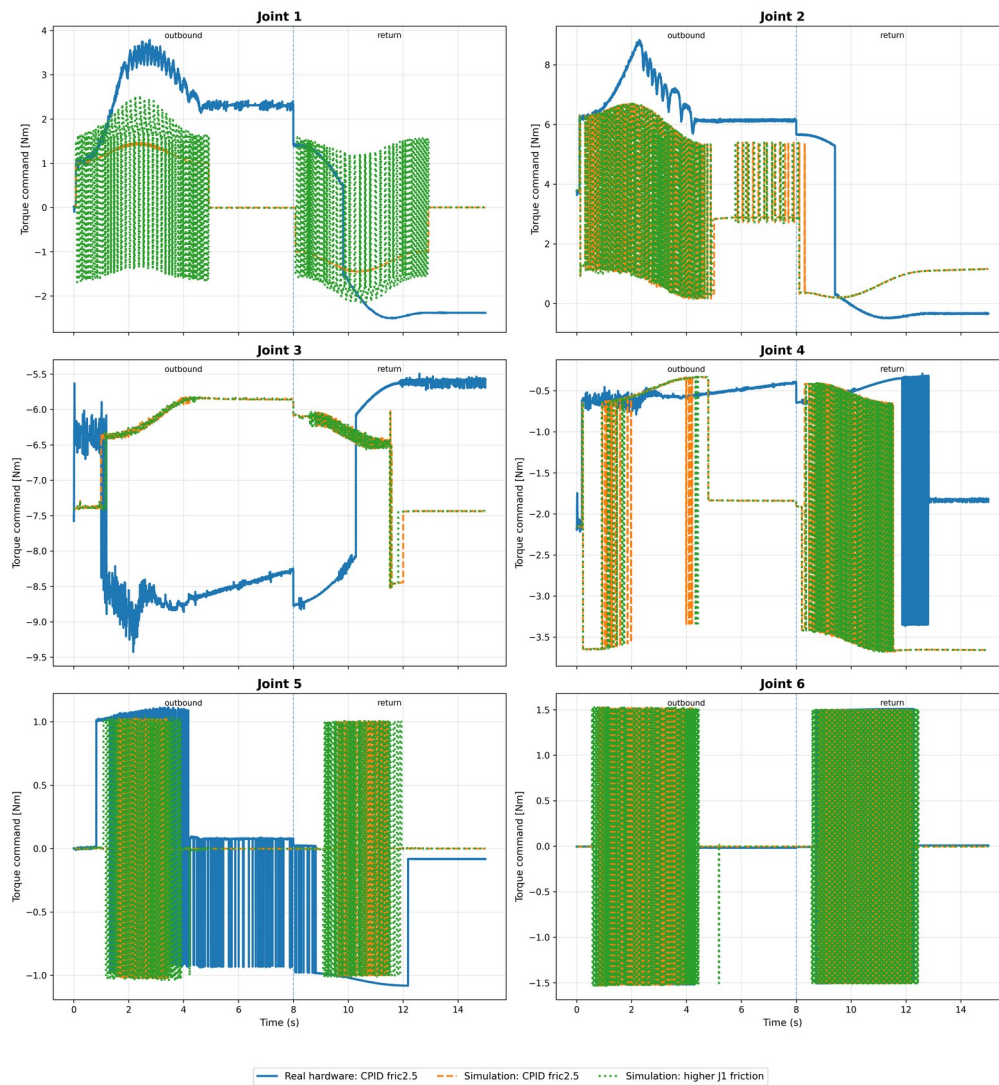
Three-case position comparison.

Forward-Pose CPID Test: Joint Position Error Comparison



Three-case position-error comparison.

Forward-Pose CPID Test: Joint Torque Comparison



Three-case torque comparison.

8.7 Benchmark meaning

The benchmark supports the main project story:

validated gripper-equivalent dynamics model + CPID torque control + friction/damping compensation + control-pa

It shows that Gazebo tracking can be good when friction and damping are compensated. It also shows that SDK LOWCMD has an internal feedback advantage over pure external torque control.

9. Real-Hardware CPID Tests

9.1 Purpose

The real-hardware tests evaluate whether the computed PID torque controller can transfer from simulation to the physical Z1 arm. The goal was not aggressive motion or perfect tracking; the goal was to test meaningful motion under safety limits and identify practical transfer limits.

9.2 Real-arm controller settings

Parameter	Value
Controller	computed_pid_friction_model
Dynamics mode	analytic
K_p	[64, 100, 100, 60, 64, 100]
K_d	[13, 16, 16, 14, 13, 16]
K_i	[0, 0, 0, 20, 0, 0]
Damping compensation	[1, 2, 1, 1, 1, 1]
Friction compensation	[1, 2.5, 1, 1.5, 1, 1.5]
Torque limit	[5, 12, 12, 10, 3, 3] Nm

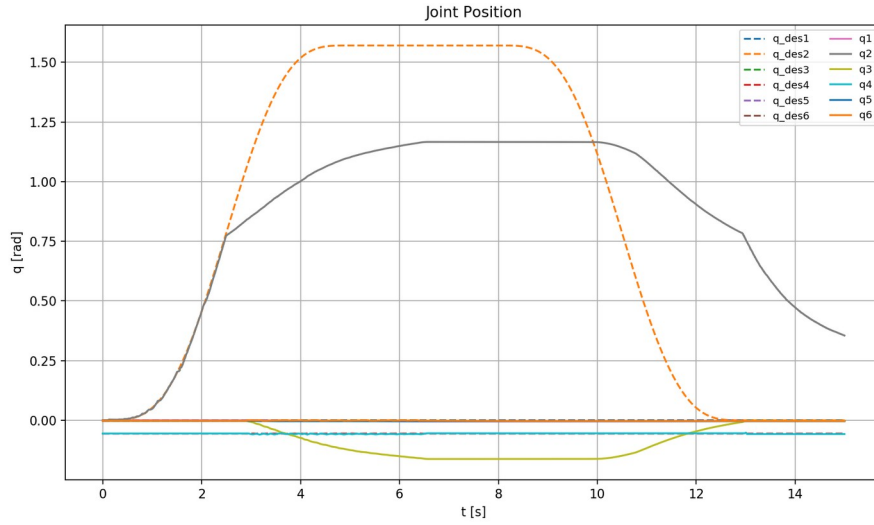
9.3 Real J2/J3 CPID validation test

A representative real-arm validation test commanded the arm toward

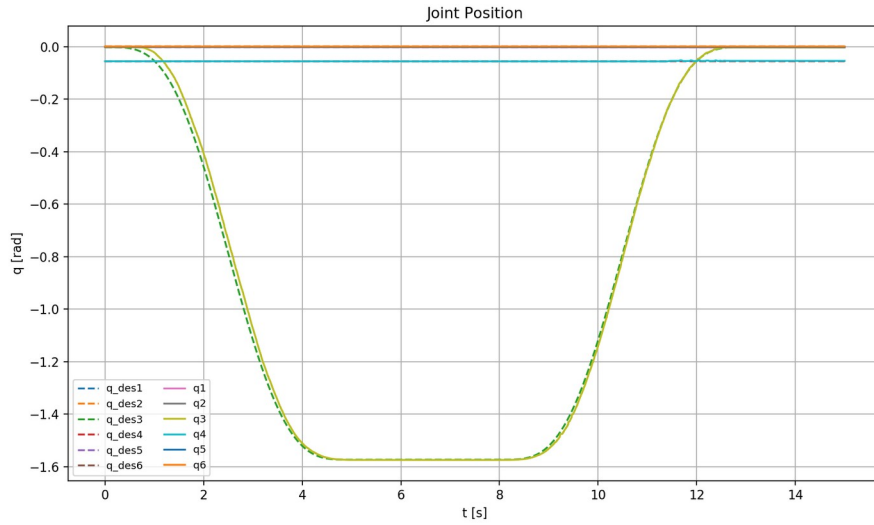
$$q_d = [0, 1.5708, -1.5708, -0.074, 0, 0].$$

This test mainly evaluates J2 and J3, which carry much of the arm/gripper load. During the outbound phase, the controller moved the physical arm close to the desired J2/J3 configuration. The maximum commanded torques were approximately

$$|\tau|_{max} = [1.00, 10.00, 10.00, 4.04, 1.00, 1.53] \text{ Nm}.$$



Real J2 CPID motion.



Real J3 CPID motion.

This result shows that the CPID controller and gripper-equivalent model can generate meaningful real-arm motion.

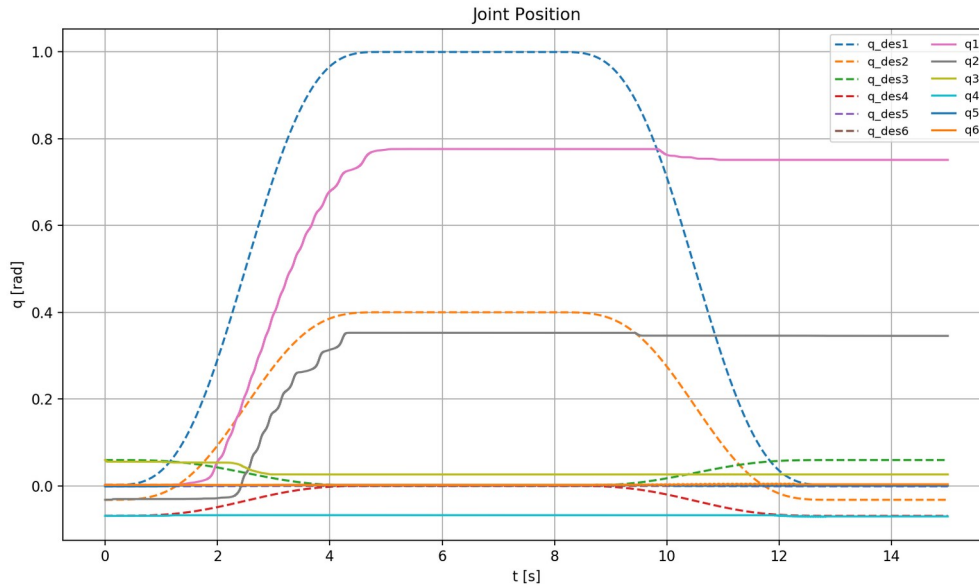
9.4 Real forward-pose CPID test

A more difficult real-hardware test used a forward-pose target approximately

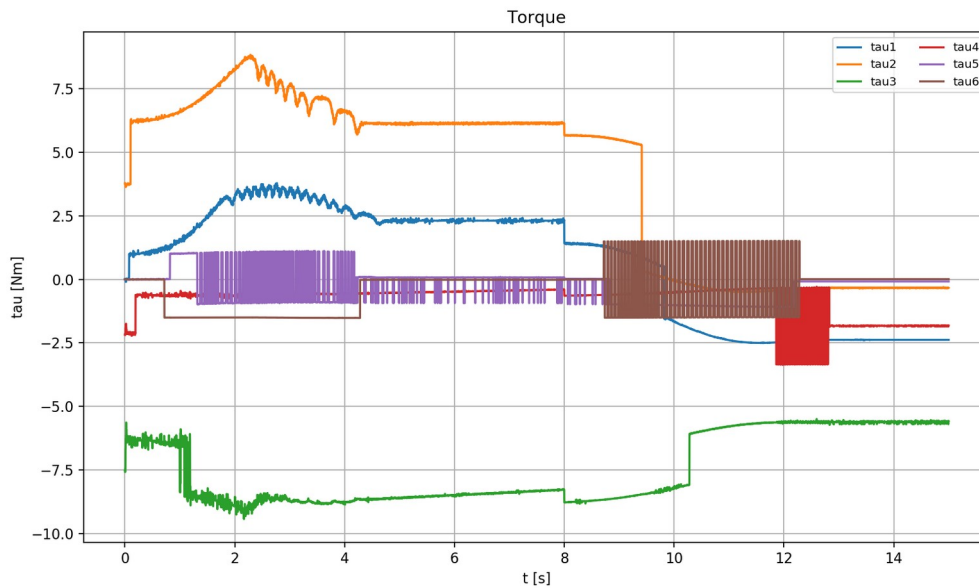
$$q_d = [1.0, 0.4, 0, 0, 0, 0].$$

The arm moved substantially toward the target, but it did not fully reach the desired pose and did not return cleanly. The maximum commanded torque during this run was approximately

$$|\tau|_{max} = [3.78, 8.83, 9.43, 3.36, 1.11, 1.52] \text{ Nm}.$$



Real forward-pose CPID joint position.



Real forward-pose CPID torque command.

The large return-phase residual errors on J1 and J2 indicate real friction/deadband and direction-dependent effects not fully captured by the simplified Gazebo model.

9.5 Why further real tests were stopped

Further real-hardware testing was stopped for safety. Increasing friction compensation could help overcome static friction, but it could also create overshoot, oscillation, heating, or unsafe motion. The project already obtained enough evidence to identify the next barrier: reliable real-arm friction/deadband identification.

10. Discussion: Model Validity, Friction, and Control-Path Limits

10.1 Main interpretation

The central result is that the gripper-equivalent analytic dynamics model is a valid base for model-based torque control of the Unitree Z1 arm. The model is implemented in code, uses explicit Z1 parameters, and is validated against Unitree SDK inverseDynamics. The same model structure supports Gazebo trajectory tracking when friction and damping compensation are included.

The main practical limitation is the gap between ideal rigid-body dynamics and the real actuator/control system. In practice, the Z1 torque-control system includes friction, damping, deadband, torque limits, motor current behavior, feedback delay, and SDK-vs-pure-torque architecture differences.

10.2 Why CPID is harder than AugPD in the external torque loop

Augmented PD applies feedback directly as joint torque:

$$\tau_{AugPD} = M(q)\ddot{q}_d + C(q, \dot{q})\dot{q}_d + N(q) + K_p e + K_d \dot{e} + \tau_{fric}.$$

Computed PID first converts feedback into desired acceleration and then multiplies by the mass matrix:

$$\tau_{CPID} = M(q)(\ddot{q}_d + K_d \dot{e} + K_p e + K_i e_{int}) + C(q, \dot{q})\dot{q}_d + N(q) + \tau_{fric}.$$

This makes CPID more directly model-based, but also more sensitive to model mismatch, delay, and friction errors. In an external Python/ROS/file bridge loop, this sensitivity becomes important.

10.3 Simulation-to-real gap

The real-arm experiments showed a larger gap than the Gazebo simulations. This is expected because the physical robot has stronger static friction and gearbox resistance, actuator deadband, motor current behavior, torque filtering, cable drag, thermal limits, and external-loop jitter.

10.4 Why friction compensation is both necessary and risky

Friction compensation is necessary because the ideal rigid-body model does not include dry friction or deadband. But excessive Coulomb compensation can cause overshoot and chattering around zero velocity. This creates a trade-off:

F too small $\Rightarrow$ incomplete motion ,

F too large $\Rightarrow$ oscillation and heating .

The real forward-pose test sits exactly in this trade-off.

10.5 Final discussion statement

The most important interpretation is:

The dynamics model is a valid base, but reliable real-arm CPID needs identified friction/deadband compensation

11. Future Work

11.1 PI-based static friction and deadband identification

At near-zero velocity,

$$\dot{q} \approx 0, \ddot{q} \approx 0.$$

The residual torque can be estimated as

$$\tau_{residual} \approx \tau_{cmd} - N(q).$$

A practical experiment is to move one joint slowly using a PI controller, record q , $\dot{q}$, τ_{cmd} , and $N(q)$, and compute the residual torque. Repeating this for positive and negative directions gives an estimate of static friction and deadband.

11.2 Constant-velocity damping identification

At approximately constant velocity,

$$\tau_{residual}(v) \approx F_c \text{sign}(v) + Bv + \tau_{offset}.$$

Running each joint at several small positive and negative velocities allows F_c and B to be identified from residual torque versus velocity.

11.3 Smooth and direction-dependent friction compensation

A smoother compensation can replace the discontinuous sign function with

$$\tau_{friction} = F_c \tanh\left(\frac{\dot{q}}{v_s}\right).$$

Direction-dependent friction values F_c^{+} and F_c^{-} should also be considered because real joints may not be symmetric in positive and negative directions.

11.4 Anti-windup for CPID

Future CPID should include anti-windup. A simple method is conditional integration:

$$\dot{e}_{int} = \begin{cases} e, & |\tau_{cmd}| < \tau_{max}, \\ 0, & |\tau_{cmd}| \geq \tau_{max}. \end{cases}$$

This prevents the integral term from growing when the actuator is already saturated.

11.5 Faster low-level implementation

A future implementation should move the controller closer to the low-level loop, ideally in C++ or through a faster SDK-based torque path. It should avoid file-based command exchange, reduce timing jitter, and log the actual control-loop period.

11.6 Improved Gazebo model

The Gazebo model should be updated with identified Coulomb friction, viscous damping, direction asymmetry, deadband, torque saturation, and delay effects. This would make simulation more predictive of real-arm behavior.

12. Conclusion

This project developed and evaluated a model-based torque-control framework for the Unitree Z1 robot arm. The main focus was the construction and validation of a gripper-equivalent analytic dynamics model and its use inside computed PID torque control.

The first major result is that the dynamics model was made explicit and reproducible. The model uses the Z1 link masses, centers of mass, inertia tensors, joint axes, and joint points, with an SDK/gripper-equivalent final-link assumption. This assumption is necessary because the real arm used in the project has the gripper installed.

The second major result is that the analytic dynamics implementation follows standard open-chain robot dynamics. The code computes twist exponentials, adjoint transformations, link Jacobians, $M(q)$, dM/dq , $C(q, \dot{q})$, and $N(q)$. These functions are mapped directly to the corresponding formulas from Murray, Li, and Sastry.

The third major result is that the gripper-equivalent analytic model agrees closely with Unitree SDK inverseDynamics. This supports the use of the model for computed torque and computed PID control.

The fourth major result is that computed PID torque control can be implemented using the validated dynamics model:

$$\ddot{q}_{cmd} = \ddot{q}_d + K_d(\dot{q}_d - \dot{q}) + K_p(q_d - q) + K_i \int (q_d - q) dt,$$
$$\tau = M(q)\ddot{q}_{cmd} + C(q, \dot{q})\dot{q} + N(q) + \tau_{fric}.$$

The fifth major result is that friction and damping compensation are essential. The ideal rigid-body equation does not include Coulomb friction, viscous damping, actuator deadband, gearbox resistance, or torque filtering. Therefore, friction compensation should be interpreted as practical actuator compensation added on top of the rigid-body model.

The sixth major result is that SDK LOWCMD and pure external torque control are structurally different. SDK LOWCMD includes internal position and damping feedback, while the project pure-torque bridge applies all feedback externally.

The real-hardware tests showed that CPID with the gripper-equivalent analytic model can produce meaningful real-arm motion under safety limits. However, larger forward-pose motion exposed strong return-phase limitations, especially on J1 and J2. These limitations are consistent with real-arm friction, deadband, torque limits, and external-loop bandwidth limits.

For safety, real-hardware testing was stopped before increasing friction compensation or gains aggressively. The project therefore identifies the next necessary step: systematic friction and deadband identification.

The final conclusion is:

The gripper-equivalent Z1 analytic dynamics model is a valid base for computed PID torque control, but reliable

Appendix A: Full Z1 Model Parameters

A.1 Gazebo no-gripper URDF parameters

These values are from the generated Gazebo URDF. In this generated file, the optional Unitree gripper is not present.

Link	Mass [kg]	COM xyz [m]	Inertia tuple [kg m ²]
link00	0.47247481	(-0.00334984, -0.00013615, 0.02495843)	(0.00037937, -3.5e-07, -1.037e-05, 0.00041521, -9.9e-07, 0.00053066)
link01	0.67332551	(2.47e-06, -0.00025198, 0.02317169)	(0.00128328, -6e-08, -4e-07, 0.00071931, 5e-07, 0.00083936)
link02	1.19132258	(-0.11012601, 0.00240029, 0.00158266)	(0.00102138, 0.00062358, 5.13e-06, 0.02429457, -2.1e-06, 0.02466114)
link03	0.83940874	(0.10609208, -0.00541815, 0.03476383)	(0.00108061, -8.669e-05, -0.00208102, 0.00954238, -1.332e-05, 0.00886621)
link04	0.56404563	(0.04366681, 0.00364738, -0.00170192)	(0.00031576, 8.13e-05, 4.091e-05, 0.00092996, -5.96e-06,

Link	Mass [kg]	COM xyz [m]	Inertia tuple [kg m ²]
link05	0.38938492	(0.03121533, 0, 0.00646316)	0.00097912) (0.00017605, 4e-07, 5.689e-05, 0.00055896, -1.3e-07, 0.0005386)
link06	0.28875807	(0.0241569, -0.00017355, -0.00143876)	(0.00018328, 1.22e-06, 5.4e-07, 0.0001475, 8e-08, 0.0001468)

A.2 Project analytic gripper-equivalent parameters

Project link	Mass [kg]	COM xyz [m]	Inertia tuple [kg m ²]
link01	0.67332551	(2.47e-06, -0.00025198, 0.02317169)	(0.00128328, -6e-08, -4e-07, 0.00071931, 5e-07, 0.00083936)
link02	1.19132258	(-0.11012601, 0.00240029, 0.00158266)	(0.00102138, 0.00062358, 5.13e-06, 0.02429457, -2.1e-06, 0.02466114)
link03	0.83940874	(0.10609208, -0.00541815, 0.03476383)	(0.00108061, -8.669e-05, -0.00208102, 0.00954238, -1.332e-05, 0.00886621)
link04	0.56404563	(0.04366681, 0.00364738, -0.00170192)	(0.00031576, 8.13e-05, 4.091e-05, 0.00092996, -5.96e-06, 0.00097912)
link05	0.38938492	(0.03121533, 0, 0.00646316)	(0.00017605, 4e-07, 5.689e-05, 0.00055896, -1.3e-07, 0.0005386)
link06 equivalent	1.09473306147355	(0.023210304542, -0.000363250494, 0.002026681669)	(0.00336754278998 6, -1.884722331677e-0 5, 0.00028294370740

Project link	Mass [kg]	COM xyz [m]	Inertia tuple [kg m ²]
			01, 0.002529152661291, 2.211249714817e-05, 0.002713697251443)

A.3 Optional gripper parameters

Optional body	Mass [kg]	COM xyz [m]	Inertia tuple [kg m ²]
gripperStator	0.52603655	(0.04764427, -0.00035819, -0.00249162)	(0.00038683, -3.59e-06, 7.662e-05, 0.00068614, 2.09e-06, 0.00066293)
gripperMover	0.27621302	(0.01320633, 0.00476708, 0.00380534)	(0.00017716, 1.683e-05, -1.786e-05, 0.00026787, 2.62e-06, 0.00035728)

The simple mass sum link06 + gripperStator + gripperMover is 1.09100764 kg. The project equivalent final-link mass is 1.09473306147355 kg.

Appendix B: Book Equation Reference Table

Report use	Equation number
Hat map	Eq. (2.4)
Cross product as matrix multiplication	Eq. (2.5)
Rotation exponential	Eq. (2.9), Eq. (2.14)
Revolute twist matrix	Eq. (2.26)
Wedge and vee operators	Eq. (2.30), Eq. (2.31)
Rigid-body exponential map	Eq. (2.36)
Adjoint matrix	Eq. (2.58)
Link kinetic energy	Eq. (4.17)
Total kinetic energy	Eq. (4.18)
Manipulator inertia matrix	Eq. (4.19)
Lagrange dynamics	Eq. (4.21)
Christoffel term	Eq. (4.22)

Report use	Equation number
Lie bracket	Eq. (4.26)
Adjoint chain	Eq. (4.27)
Transformed inertia	Eq. (4.28)
Explicit inertia matrix formula	Eq. (4.29)
Explicit derivative of inertia	Eq. (4.30)
Manipulator dynamics	Eq. (4.47)
Computed torque control law	Eq. (4.49)
Computed torque error dynamics	Eq. (4.50)
Basic PD control	Eq. (4.51)
Modified PD with feedforward	Eq. (4.53)

Appendix C: Reproducibility Notes

The project repository is public at:

https://github.com/iceswordIV/torque_control

Important code files include:

- `z1_project/z1_analytic_dynamics.py`: analytic dynamics model and Z1 parameters.
- `z1_project/test_controller.py`: diagnostic torque controllers, CPID, friction compensation.
- `z1_project/compare_sdk_dynamics.py`: SDK inverseDynamics comparison.
- `z1_project/analyze_final_controller_comparison.py`: result analysis.
- `reference/z1_description/xacro`: Z1 description parameters, including optional gripper bodies.

References

Murray, R. M., Li, Z., and Sastry, S. S. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.

Unitree Robotics. Unitree Z1 robot arm documentation, SDK examples, and Z1 description files used in the public project repository.

Project code repository: https://github.com/iceswordIV/torque_control